```
==14817== For counts of detected and suppressed
errors, rerun with: -v
==14817== ERROR SUMMARY: 0 errors from 0 contexts
(suppressed: 15 from 8)
```

The leak summary shows that there was definitely a memory leak. You can use the options *–tool=memcheck --leak-check=full* to obtain a more detailed output. Refer to the documentation in the *valgrind-3.5.0/docs* directory to get more information.

You might wonder what's the use of running *valgrind* on test binaries rather than source binaries? It's handy when the source is cross-compiled to run on different target architectures, and thus not realistic to test it frequently. In our *test.c* example, the release executable is intended for the ARM architecture:

```
bash>make release
bash>file release.bin
release.bin: ELF 32-bit LSB executable, ARM,
version 1 (ARM), for GNU/Linux 2.6.4, dynamically
linked (uses shared libs), for GNU/Linux 2.6.4, not
stripped
```

Note: A discussion of cross-compilation is beyond the scope of this article, but I've mentioned a reference at the end of the article that will provide you with more information, if you're interested.

You can also try the code coverage and *valgrind* check on the source binary without building it into the unit testing target—you can build with the *release_cov_valgrind* make target:

```
bash>make release_cov_valgrind
```

Useful definitions

Native compilation: Building executables for the same platform as the one on which the compiler is run, to compile the code.

Cross-compilation: Creation of executables for a target platform other than the one on which the compiler is run (which is the build platform)

Here are a few exciting ideas before signing off:

1. You can implement the idea of unit testing in coding contests, to validate and compare the submitted results. Evaluation is made easier by automating the testing of the submitted code.
2. This could be helpful in examinations—for teachers who are assessing students' program submissions.
3. A logical mix of unit testing, code coverage, memory leak and error checking is a valuable validation, verification and code quality measurement, especially in corporate projects.

Finally, I would like to acknowledge all the people who contributed to the UnitTest++, gcov, lcov, and *valgrind* open source projects, and thank them for their efforts. END

Links

http://sourceforge.net/projects/unittest-cpp
http://ltp.sourceforge.net/coverage/lcov.php
http://valgrind.org/downloads/
http://sourceforge.net/projects/ltp/files/Coverage%20Analysis/LCOV-1.8/lcov-1.8.tar.gz/download
http://www.tazenda.demon.co.uk/phil/arm-tools.html
http://www.ailis.de/~k/archives/19-ARM-cross-compiling-howto.html

By: Raghavendra K T

Raghavendra is a Linux enthusiast and a software developer. He has been working with IBM for the last 4 years, in the embedded systems solution department. You can reach him at raghavendra.kt@gmail.com